



High-Level Synthesis Directives Design Optimization via Large Language Model

XUFENG YAO, Computer Science and Engineering, CUHK, Hong Kong, Hong Kong

WENQIAN ZHAO, The Chinese University of Hong Kong, Hong Kong, Hong Kong

QI SUN, Zhejiang University, Hangzhou, China

CHENG ZHUO, Zhejiang University, Hangzhou, China

BEI YU, CUHK, Hong Kong, Hong Kong

High-level synthesis is an effective methodology that accelerates early-stage circuit design. The optimization of HLS directives has been a critical yet challenging endeavor, with prevailing research primarily concentrating on custom feature engineering and dedicated model designs. However, these conventional approaches often fall short of fully harnessing the intricate latent information embedded within raw HLS directives, potentially limiting the scope and efficiency of optimization processes. In response to these challenges, this article pioneers the integration of large language model (LLM) into the HLS optimization workflow, leveraging their capabilities as both sophisticated feature extractors and autonomous agents. This application of LLM marks a significant departure from traditional methods, introducing a more nuanced and effective strategy for navigating the complex landscape of HLS directive optimization, enabling a more efficient exploration of the design space and prioritization of search strategies. Specifically, our approach makes a significant improvement to the Pareto frontier in directive design, enabling a more rapid and efficient design space exploration. This demonstrates not only an increase in optimization performance but also a decrease in computational overhead, thereby promising significant time savings in the circuit design process. This work not only enhances the current state of HLS directive optimization but also makes new avenues for the application of language models in the field of EDA. Our work makes the following key achievements: We propose an LLM-based framework for effective HLS directives design space exploration; We utilize the prior knowledge of LLM and fine-tune an LLM for HLS directives optimization; Empirical results demonstrate this LLM-based approach's effectiveness. Specifically, we obtain 15% improvement on the normalized ADRS metric, demonstrating superior performance with limited sampling steps compared with current leading algorithms.

CCS Concepts: • **Hardware** → **High-level and register-transfer level synthesis**; • **Computer systems organization** → **Reconfigurable computing**; • **Computing methodologies** → **Search methodologies**; **Neural networks**; **Natural language processing**;

Additional Key Words and Phrases: High-level synthesis (HLS), FPGA, synthesis directives, design space exploration (DSE), bayesian optimization, gaussian processes, large language models (LLMs), pareto optimization, power-performance-area (PPA)

Authors' Contact Information: Xufeng Yao, Computer Science and Engineering, CUHK, Hong Kong, Hong Kong; e-mail: yxf12fdu@gmail.com; Wenqian Zhao, The Chinese University of Hong Kong, Hong Kong, Hong Kong; e-mail: wqzhao@cse.cuhk.edu.hk; Qi Sun, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: qisunchn@zju.edu.cn; Cheng Zhuo, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: czhuo@zju.edu.cn; Bei Yu (corresponding author), CUHK, Hong Kong, Hong Kong; e-mail: byu@cse.cuhk.edu.hk.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2025 Copyright held by the owner/author(s).

ACM 1084-4309/2025/09-ART78

<https://doi.org/10.1145/3747291>

ACM Reference Format:

Xufeng Yao, Wenqian Zhao, Qi Sun, Cheng Zhuo, and Bei Yu. 2025. High-Level Synthesis Directives Design Optimization via Large Language Model. *ACM Trans. Des. Autom. Electron. Syst.* 30, 5, Article 78 (September 2025), 24 pages. <https://doi.org/10.1145/3747291>

1 Introduction

High-level synthesis (HLS) [1–3] plays an important role in VLSI design automation, enabling the translation of high-level programming languages such as C/C++ into low-level **hardware description languages (HDLs)** like Verilog or VHDL. This process reduces the amount of code required and speeds up the verification process. In HLS, directives serve as pivotal guides for converting high-level programming descriptions into specific hardware implementations. These directives inform the synthesis process on various fronts, including the parallelization of tasks, allocation of memory, and deployment of computational resources. By incorporating HLS directives into the design, we can optimize the implementation in terms of power, performance, and area [4]. The intricacy of HLS lies in the variability of the output hardware architectures, which can significantly differ based on the configurations of these directives, leading to diverse performance outcomes. The crux of designing effective HLS directives is to identify configurations that maximize design objectives within a vast design space filled with potential directive combinations. Key performance metrics typically include minimizing power consumption, reducing computational delay, and optimizing resource utilization. Figure 1 shows an example of HLS directives. The selection of different directive configurations has a significant impact on the efficiency and quality of resulting RTL designs. Nevertheless, due to the high running cost, it's not feasible to perform an exhaustive search in modern designs.

Numerous studies have devoted considerable attention to optimizing HLS from various perspectives [5–8]. One practical solution to find optimal directive configurations is black-box optimization: taking it as a **design space exploration (DSE)** problem, where the goal is to develop an efficient algorithm to perform trial runs across the search space of parameters, seeking the optimal configurations that meet the requirements of **power-performance-area (PPA)**. Figure 2 presents an example of HLS design flow. The design optimization seeks to find the Pareto optimal frontier of designs, representing the best tradeoff solutions between conflicting design objectives. However, the complexities of HLS directive design lead to a supra-linear growth in search space with the number of directive options, resulting in a time-consuming and challenging search for Pareto optimal designs. The entire DSE flow is automated, designed to efficiently generate optimal design points, which include both HLS code and corresponding directives. Initially, the HLS source code along with the associated directives is input into the LLM, as detailed in Section 3. This setup enables the LLM to effectively analyze and understand the HLS code, subsequently identifying and extracting informative design points through its sophisticated comprehension capabilities. These chosen design points, which comprise specific HLS code and directives, are then processed using FPGA design tools to generate detailed PPA reports. Upon receipt of the PPA report, we meticulously extract the relevant PPA values and use these metrics to fine-tune the LLM, enhancing its predictive accuracy and efficiency. This cycle of feedback and refinement is repeated several times, allowing us to iteratively converge on the desired design points that meet our predefined optimization criteria.

Motivation 1: Fine-tune LLM as a feature extractor. Many efforts have been made to tackle this problem. A typical solution is to leverage model-based methods to estimate the performance of HLS, and then a better HLS directive is estimated by the increasing precision of the model. Some typical models, including Linear Regression, Artificial Neural Network, GNN, and Gradient Boosting [9–11], are proposed to address the problem. Conventionally, these methods convert raw directives into numerical values by customized feature engineering, which may overlook potential useful information.

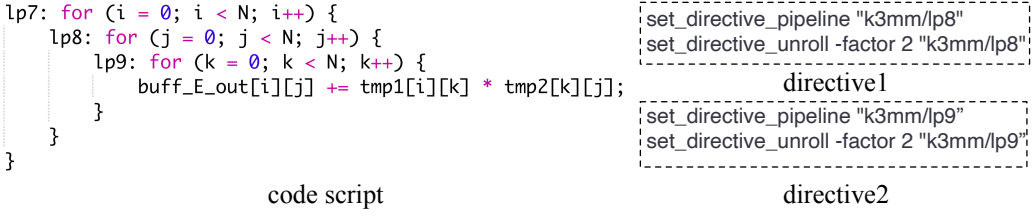


Fig. 1. HLS pseudo-codes and different directives.

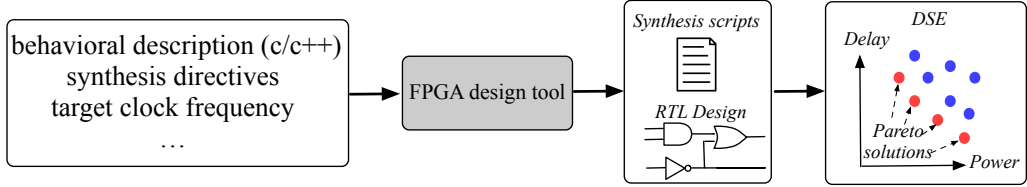


Fig. 2. HLS design flow. A behavioral description, synthesis directives and related constraints are given to the design tool, which enables the tool to generate RTL design and synthesis scripts. The goal of directive design optimization is to find Pareto solutions of objectives (e.g., power and delay) across the whole design space.

For instance, “set_directive unroll factor=(2, 5, 10)” is transformed into numerical factors, such as (0, 0.375, 1.0) after normalization [12]. This transformation does not encode contextual relationships and hierarchical information, thus it may struggle to discern complex patterns with limited samples. On the other hand, LLM is inherently good at digging for useful hidden semantic information and reasoning complex relationships. A direct approach is to send raw directives to LLM and extract hidden feature outputs such that we can obtain informative features without manual feature engineering or complex model designs.

Motivation 2: Fine-tune LLM as a directives comparator. Another important motivation is that LLM was powered with prior knowledge on which candidate point is possibly feasible to improve performance effectively and which is not. A simple example is visualized in Figure 1 where two similar loop-unrolling directives are provided. A conventional black-box DSE model will consider them as two distinct samples to explore. Meanwhile, an LLM with prior knowledge can understand to some extent that: “directive 2” applying loop pipelining/unrolling the innermost loop can help eliminate loop control overhead and expose more parallelism to the HLS tool. On the other hand, “directive 1” requires overlapping of the inner loop, which requires higher duplicate hardware resources and may not be effective under hardware limitation. With awareness and understanding of such characteristics, LLM-based approaches can avoid infeasible choices and make a smart sampling policy.

This work proposes an LLM-based solution on top of Bayesian Optimization and Gaussian Process. Key innovations include a directives comparison algorithm drawing on LLM’s prior knowledge, an automated and effective feature extraction of raw directives via LLM and, a coarse-to-fine sampling pipeline. Furthermore, we introduce a new LLM update pipeline via synthesis feedback to enhance the effectiveness of LLM in HLS directives optimization. Our contributions are summarized as follows:

- By exploiting rich information within directives, we propose an LLM-based framework for HLS directives design optimization.
- We take advantage of the prior knowledge of LLM to provide a comparison between directives which helps to improve the efficiency of DSE. A novel LLM update pipeline from synthesis feedback has been proposed to fine-tune LLM.

- A hierarchical two-stage sampling approach is introduced to offer a coarse-to-fine sampling to identify promising regions for effective search.
- Extensive experiments demonstrate the effectiveness of the proposed method.

2 Preliminaries

2.1 HLS Design Space Exploration and Challenge

HLS directives can be represented as a vector $\mathbf{x} \in \mathcal{R}^d$ in the design space $\mathcal{X}$. The primary objective of DSE is to identify the configuration from $\mathcal{X}$ that achieves the optimal performance f . Typically, the performance is a combination of multiple objectives $\mathbf{y} = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_N(\mathbf{x})]$. Multiple objectives often have conflicts with each other, which can be practically transformed into a problem that aims to find a Pareto optimal set.

Definition 1 (Pareto optimality). In an N -dimension minimization problem, an objective vector $f(\mathbf{x})$ is said to dominate $f(\hat{\mathbf{x}})$ if

$$\begin{aligned} \forall i \in [1, N], f_i(\mathbf{x}) &\leq f_i(\hat{\mathbf{x}}) \text{ and} \\ \exists j \in [1, N], f_j(\mathbf{x}) &< f_j(\hat{\mathbf{x}}). \end{aligned} \quad (1)$$

Nevertheless, the evaluation of HLS performance is extremely time-consuming because the following logic synthesis and implementation steps are necessary to attain accurate performance. More accurate evaluation requires longer low-level implementation flows to omit detailed information. Such a bottleneck is hard to tackle even if an efficient search strategy is available. LLM with context understanding ability and sufficient domain knowledge may have the inherent evaluation ability to evaluate design performance or at least pre-determine some catastrophic candidates, which has the potential to enhance the searching efficacy.

2.2 Bayesian Optimization and Gaussian Process

Bayesian optimization (BO) is a sequential strategy for optimization of black-box functions that are expensive to evaluate. The optimization procedure consists of alternating between two steps. (1) Surrogate model (e.g., Gaussian process) involves using training data (e.g., directives) to construct a probabilistic model of objective functions (e.g., PPA), which can provide a quantified estimate of performance and uncertainty. (2) Acquisition function (e.g., Upper Confidence Bound [13]) is formulated to determine the next sampling point based on the surrogate model. It serves as a heuristic guide to balance exploration and exploitation in DSE tasks.

Gaussian process (GP) regression is a non-parametric method to model the objective function. It's defined as

$$f(\mathbf{x}) \sim \text{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')), \quad (2)$$

where GP indicates for Gaussian process. $f(\mathbf{x})$ represents objectives, $m(\mathbf{x})$ and $k(\mathbf{x}, \mathbf{x}')$ represent mean and variance function, respectively. Given observed directives and related synthesis results data $D\{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_n, \mathbf{y}_n)\}$, the observations $\mathbf{y}$ are related to the function values f by $\mathbf{y} = f(\mathbf{x} + \epsilon)$ where ϵ is the noise term which often assumed to be Gaussian: $\epsilon \sim \mathcal{N}(0, \sigma_n^2)$. Assume n targeted values $\mathbf{y}$ are already evaluated from the training set $\mathbf{X}$. For new directive configuration $\mathbf{x}_*$ and related objectives f_* , the joint distribution of observed objectives $\mathbf{y}$ and f_* is given by

$$\begin{bmatrix} \mathbf{y} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I} & K(\mathbf{X}, \mathbf{x}_*) \\ K(\mathbf{x}_*, \mathbf{X}) & K(\mathbf{x}_*, \mathbf{x}_*) \end{bmatrix}\right), \quad (3)$$

where $K(\mathbf{X}, \mathbf{X})$ is covariance matrix with elements $k(\mathbf{x}_i, \mathbf{x}_j)$, $K(\mathbf{X}, \mathbf{x}_*)$ is the covariance matrix between training inputs and test inputs. $K(\mathbf{x}_*, \mathbf{x}_*)$ is the covariance matrix between test input, σ_n^2

is the noise variance that is used to prevent over-fitting. When a new design point is sampled from the design space, we will update $f(\mathbf{x})$ accordingly.

2.3 Large Language Model

Recently, the decoder-only transformer-based LLM has demonstrated great capabilities in various domains [14–17]. In the context of decoder-only transformer-based architectures in large language models, as illustrated in Figure 3(a), the architecture prominently features a sequence of layers, with the multi-head self-attention mechanism standing out as a pivotal component, along with other components such as RMS Norm [18], SwiGLU [19]. The inputs are first transformed into embeddings and pass through a layer normalization step (RMS Norm) for normalization. Next, they proceed to the multi-head self-attention mechanism with multiple query (Multi-Query attention), where the sequence is concurrently processed through different attention heads to capture various contextual relationships. The output of this attention is then combined and again normalized with RMS Norm. Subsequently, the data flows through a **feed-forward network (FFN)**, employing a SwiGLU activation function, which further refines the features extracted by the attention mechanism. The output of this FFN is finally added to the original input through a residual connection, completing one pass through the transformer block. This process iteratively repeats across multiple such blocks to progressively refine the representation of the input, ultimately leading to the generation of the next element in the sequence.

Figure 3(b) illustrates the variants of self-attentions of modern LLM architectures. Among them, Multi-Head attention is the most widely used attention mechanism that introduced in Reference [20], which is given by

$$\text{MultiHead}(Q, K, V) = \text{Concat}(H_1, H_2, \dots, H_n)W^O, \quad (4)$$

where $Q = QW^Q$, $K = KW^K$ and $V = VW^V$, W^Q , W^K , W^V are corresponding projection matrix that maps Q , K and V to d_q , d_k , d_v dimensions, respectively. W^O is another project matrix. Each head $H_i, i \in \{1, 2, \dots, n\}$ is the output of a single scaled dot-product attention head, which can be formulated as

$$H_i = \text{Softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i. \quad (5)$$

Multi-Query Attention [21] is an adaptation of the attention mechanism that allows multiple queries to be processed simultaneously. While not fundamentally different in form from the equations above, the distinction lies in how Q is structured or interpreted. In multi-query attention, Q consists of multiple sets of queries processed against the same K and V . Group-Query Attention [22] is a good compromise between quality and speed. It organizes queries into groups, where each group might have different processing or weighting schemes. The Group-Query Attention can be given by

$$\text{Attention}_g(Q_g, K_g, V_g) = \text{Softmax}\left(\frac{Q_g K_g^\top}{\sqrt{d_{kg}}} V_g\right). \quad (6)$$

Multi-Head Attention is known for its high-quality outcomes but tends to be slow. On the other hand, Multi-Query Attention speeds things up at the expense of some quality. Group Multi-Query Attention finds a sweet spot between the two, offering a nice mix of speed and quality. This approach has become popular in the latest language models including llama2 [23] and Mistral [24].

Figure 3(c) illustrates the prediction pipeline of LLM inference stage. Given an input $\{x_1, x_2, \dots, x_n\}$, define LLM as π_θ , the next token can be given by

$$x_{n+1} = \pi_\theta(x_1, x_2, \dots, x_n), \quad (7)$$

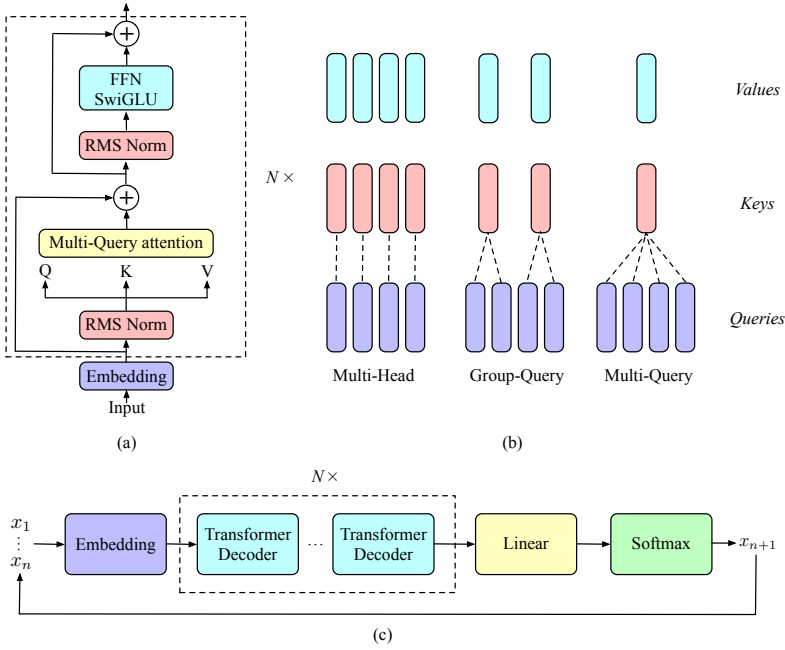


Fig. 3. (a) Transformer Decoder. (b) Variations of self-attentions. (c) Next token prediction mechanism.

where $x_i, i \in \{1, 2, \dots, n\}$ represents i -th token. After generating x_{n+1} , we concat x_{n+1} to the original input and send to LLM to generate the next token.

The development of large language model has shown great potential in hardware design [25–27], eda script generation [28], RTL code generation and debugging [29–31] and other eda applications such as [32–34]. However, these works falls short in tasks that require exploration. Previous works mainly focus on developing a domain LLM with pre-training or **Supervised Fine-Tuning (SFT)** technique, which does not consider more complicated scenarios such as DSE which contains strategy or policy learning.

Definition 2 (Thought). A “thought” in the context of LLM can be understood as the output generated by the model in response to a given prompt or input. It encompasses the model’s ability to generate coherent and contextually relevant text based on its understanding of the input and its knowledge base.

Definition 3 (Policy). A “policy” in the context of LLM is a strategy that is used to instruct or guide the model to generate a desired type of response.

On the other hand, as illustrated in Reference [35], **Reinforcement Learning from Human Feedback (RLHF)** is an important technique that aligns LLM’s generation policy with reward from a human. Typically, RLHF requires a reward model and a policy model. The reward model is supervised by human feedback which ranks the preference of different answers to the same question. The underlying goal is to get a model that takes sequences of text and returns a scalar reward that can numerically represent human preference. Then a policy model can be tuned with a “reward” weight that guides the preference of different generated answers. In this article, inspired by RLHF techniques, we aim at training LLM to calibrate their “policies” on directives optimization. Different from RLHF in the ChatGPT scenario which requires large amounts of human annotations. HLS

naturally has a wide array of synthesis results in the DSE pipeline, and we utilize these synthesis results to fine-tune our LLM.

2.4 High-Level Synthesis Design Space Exploration

HLS DSE has emerged as a crucial phase in hardware design, enabling automatic exploration and optimization of various design alternatives. Previous research has extensively investigated techniques ranging from heuristic-based approaches to advanced machine learning methods. Early efforts include [36], who introduced a divide and conquer approach for efficient exploration of large HLS design spaces. Similarly, Reference [37] proposed leveraging machine learning predictive modeling to rapidly estimate HLS outcomes, significantly accelerating the exploration process. Reference [38] combined machine learning models with simulated annealing to efficiently navigate complex HLS design spaces, demonstrating improved optimization performance. Heuristic and probabilistic approaches have also been explored. Reference [39] proposed a cluster-based heuristic approach to efficiently prune and explore large HLS design spaces, providing a balance between exploration efficiency and solution quality. Furthermore, Reference [40] introduced a probabilistic multiknob exploration technique, accelerating the search process by intelligently sampling promising design configurations. Recent research has proposed advanced optimization frameworks that leverage mathematical modeling and formal methods. Reference [41] utilized model-based optimization techniques for effectively tuning HLS directives, providing systematic and scalable optimization strategies. Reference [42] demonstrated automatic insertion of hardware pragmas using a non-linear programming approach, improving both the accuracy and efficiency of directive assignment. Reference [43] formalized loop-carried dependencies using the Coq proof assistant to enhance correctness and reliability in the HLS optimization process. Innovative compiler and transformation-based methods have also advanced the scope of HLS optimization. Reference [5] proposed polyhedral transformation techniques to expand HLS optimization opportunities, enabling more systematic and powerful optimizations. Reference [44] conducted an extensive study evaluating the state-of-the-art in HLS, highlighting the advancements and identifying key remaining optimization challenges. Reference [45] introduced AnyHLS, a partial evaluation-based framework that enhanced flexibility and optimization potential during HLS design exploration. Recent advancements have seen the emergence of deep learning methods for automated HLS optimization. [46] developed machine learning models to predict optimal loop unrolling factors, significantly improving directive optimization efficiency. Reference [47] introduced DeepFlexiHLS, a deep neural network approach that automatically generates flexible and effective HLS directives, demonstrating substantial improvements in optimization accuracy and generalization. Despite these extensive advancements, existing studies primarily rely on heuristic methods, machine learning predictive modeling, or formal modeling techniques. Our proposed work complements these existing efforts by uniquely integrating LLMs to reason about and optimize HLS directives, explicitly guided by empirical PPA metrics extracted from FPGA synthesis reports.

2.5 Large Language Models in High-Level Synthesis

The application of large models in electronic design automation is a rapidly emerging field, as demonstrated by recent advancements in hardware design [48–58]. Significant efforts have been made to leverage **large language models (LLMs)** in the domain of HLS. Reference [59] introduced foundational concepts and preliminary studies on LLM-assisted HLS design. Reference [60] and [61] explored novel LLM-based solutions for HLS debugging. HLSPilot [62] proposed an LLM-powered copilot system designed for HLS code generation and debugging tasks. Despite these advancements, there is currently no work focused on utilizing LLMs for HLS directive optimization, leaving this critical area unexplored. Several studies have further explored the integration of LLMs within

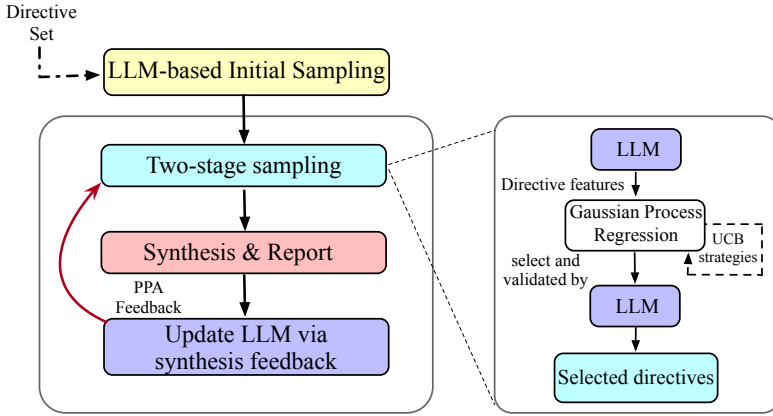


Fig. 4. Overview of the LLM-based exploration flow.

the high-level synthesis design process. Reference [63] introduced a retrieval-augmented LLM framework designed to optimize HLS designs, demonstrating improved accuracy and generalization by retrieving relevant knowledge from external sources. [64] evaluated the effectiveness of multiple contemporary LLMs for automatically generating **Register Transfer Logic (RTL)** from high-level specifications, providing empirical insights into the strengths and limitations of modern language models in hardware-centric tasks. Reference [65] performed a comprehensive benchmarking study examining various LLMs' capabilities and limitations when applied to typical HLS tasks, critically assessing their practical utility and highlighting important open challenges. [66] proposed leveraging LLMs to generate, simulate, and deploy a uniform random number generator hardware design, demonstrating the feasibility of end-to-end LLM-driven hardware development workflows. Additionally, Reference [67] introduced Agentic-HLS, an innovative LLM-based HLS system that employs agentic reasoning techniques, enabling iterative refinement and intelligent decision-making during synthesis processes. While these recent efforts significantly advance the understanding and applicability of LLMs in various aspects of the HLS domain, none explicitly focus on directive optimization informed directly by **performance, power, and area (PPA)** metrics extracted from HLS tools. Our work uniquely complements these existing studies by specifically addressing the previously unexplored yet critical area of automated directive optimization using LLM-based reasoning guided by empirical synthesis outcomes.

3 Methodologies

3.1 Overview of Framework

Figure 4 illustrates the overview of the proposed method. We incorporate LLM at each part to enhance DSE performance. In the beginning, an LLM-based initial sampling strategy is introduced to leverage the LLM's inherent ability to compare the performance of original directives. The following stage is iterative LLM tuning and DSE. Figure 5 illustrates excerpts from typical Vivado HLS reports generated after synthesis runs. These reports exhibit a consistent and structured format, thereby enabling the automated extraction of relevant performance metrics (such as latency, throughput, resource utilization, and power consumption) via custom-developed scripts. Specifically, the proposed DSE flow leverages these automated scripts to parse and extract key PPA metrics from Vivado reports systematically and seamlessly. During LLM updating, we construct a directives comparison task to guide the LLM to select better directives via synthesis feedback. During the directives sampling stage, we adopt a two-stage sampling strategy. We leverage LLM as an effective

Total On-Chip Power (W)	0.125
Design Power Budget (W)	Unspecified*
Power Budget Margin (W)	NA
Dynamic (W)	0.020
Device Static (W)	0.105
Effective TJA (C/W)	11.5
Max Ambient (C)	83.6
Junction Temperature (C)	26.4
Confidence Level	Medium
Setting File	---
Simulation Activity File	---
Design Nets Matched	NA

(a) Partial Power report

TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)
WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
0.000	0	7260	0.132
3.750	0.000	0	2607

(b) Partial Timing report

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	2052	0	53200	3.86
LUT as Logic	1284	0	53200	2.41
LUT as Memory	768	0	17400	4.41
LUT as Distributed RAM	768	0		
LUT as Shift Register	0	0		
Slice Registers	1821	0	106400	1.71
Register as Flip Flop	1821	0	106400	1.71
Register as Latch	0	0	106400	0.00
F7 Muxes	128	0	26600	0.48
F8 Muxes	0	0	13300	0.00

(c) Partial Utility report

Fig. 5. Partial vivado report results.

feature extractor for raw directives with GP. Then we bring in LLM into this flow to filter the given selected directives via the UCB algorithm and then synthesize them as feedback to update the LLM itself. The updated LLM is also utilized to further compare and select directives in the second stage. With this exploration and exploitation flow, we can search for the optimal points more effectively.

3.2 LLM-Based Initialization Sampling

Overcoming cold start is crucial in DSE because poor initialization sampling may lead to local optima. We observed that LLM is capable of comparing and ranking the performance of given directives with above-average accuracy. Inspired by the recent progress of prompt engineering like chain-of-thought [68], we propose a new “analyze then compare” prompt engineering approach for directives comparison.

Autonomous directives comparison via LLM. One problem in leveraging LLM in DSE pipeline is that it’s difficult to produce structured output due to its auto-regressive generation manner. To address this problem, we transform the comparison problem into a multi-choice question

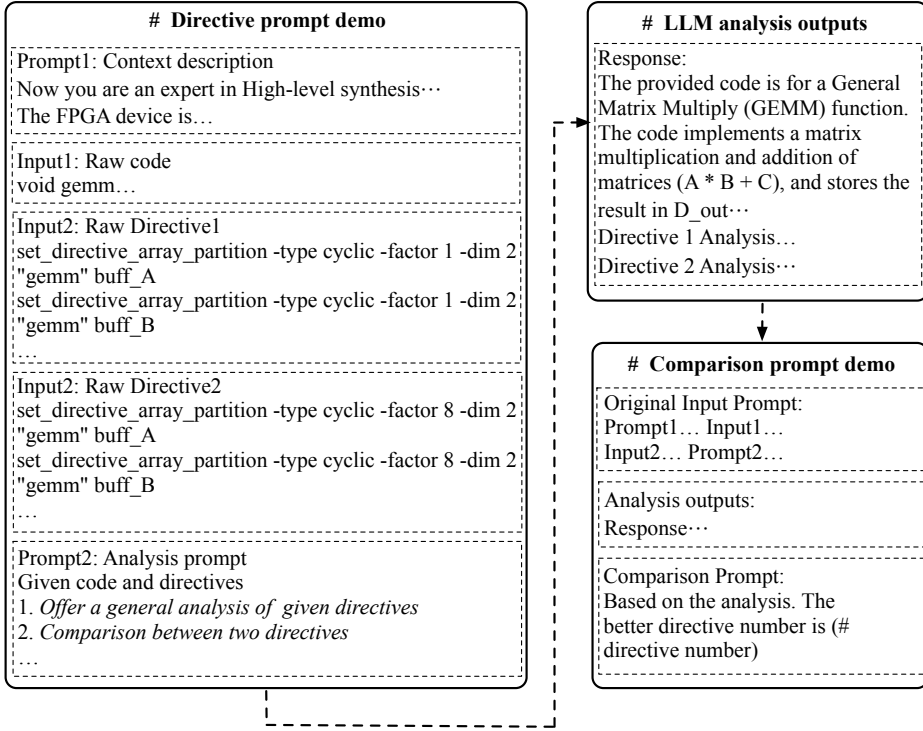


Fig. 6. An example of directive comparison input and prompt that is used to compare directives.

answering problem. For each directive comparison pair, we provide an index, e.g., directive1 into prompt and design a comparison prompt in the end to ask LLM to produce a better directive index. We only use the first generated token of LLM and select the directive index, e.g., 1 with the largest probabilities prediction. Figure 6 illustrates a prompt example designed to assist LLM in comparing different HLS directives. Initially, we integrate directive-specific prompts with the original HLS inputs into the LLM to generate a preliminary analytical thought. This initial output, termed analysis thought, encapsulates the LLM's understanding and preliminary insights about the directive's potential impact. Subsequently, this analysis thought is combined with the original inputs and prompts to construct an enriched new prompt. This iterative refinement aims to deepen the LLM's analysis by reintroducing and contextualizing the synthesized insights. The final stage involves appending a comparison prompt to this enriched prompt, specifically designed to solicit the LLM's definitive evaluation on the comparative efficacy of the directives presented. Processing this comprehensive prompt allows the LLM to deliver a refined decision, effectively guiding the optimization of HLS directives based on sophisticated reasoning and comparative analysis. This methodology leverages the LLM's advanced capabilities in parsing complex data structures and generating strategic insights, facilitating a more informed directive selection process. We define directive prompt as p_{t1} , comparison prompt demo as p_{t2} , representing the large language model as π_θ , we can express the process as follows:

$$\begin{aligned}
 th &= \{\pi_\theta(x, p_{t1}), \pi_\theta(x, p_{t1}, th_1), \dots, \pi_\theta(x, p_{t1}, th_1, \dots, th_{n-1})\}, \\
 out &= \pi_\theta(x, p_{t1}, th, p_{t2}), \\
 pr &= \text{softmax}(out), \\
 choice &= \text{argmax}(\{pr_{d_1}, pr_{d_2}\}),
 \end{aligned} \tag{8}$$

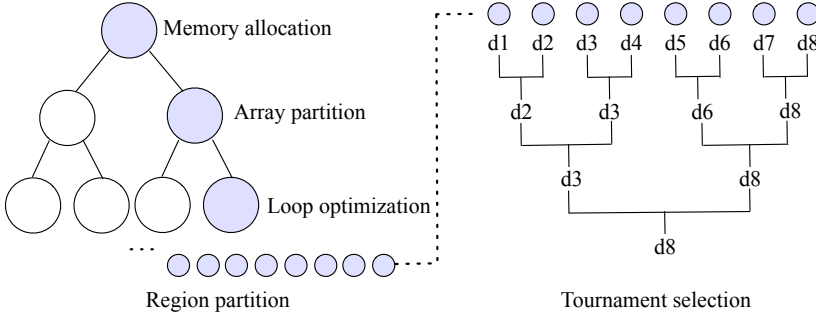


Fig. 7. Initial sampling process of one region.

where *th* represents the analysis thought, which combines all auto-regressive generated tokens. *out* is the first generated token's logits output and *pr* is the probability output, which is derived by the softmax function. The directive number index is determined by selecting the argmax of the directive number probabilities.

To select initial design points from the whole space, a direct approach is to rank the overall performance of whole directive points by LLM and select the top *k* points. However, this approach is computationally intensive for large design spaces. Additionally, cherry-picking top *k* points could lead to an imbalanced sampling of the whole design space. Ideally, to balance exploration and exploitation, we hope our sampling can span diverse regions of the whole design space while selecting the highest-performing design points within selected regions. To achieve this, we proposed a hierarchical sampling strategy. Figure 7 shows an example of sampling process (coarse-to-fine granularity) of one region. Initially, we can partition the directive by high-level parameters such as RAM pragma type and select samples from each region. RAM pragmas in HLS guide how memory is used in hardware design. At the finer partitioning, we will go further by selecting parameters such as loop unrolling factors, orders, and so on. In the following stage, we chose a tournament selection order that is widely used in sports fields such as football. The main idea is that directive design points compete against each other in a series of matches, with the losing design point being eliminated from the tournament. The winning design point advances to the next round. Specifically, for each directive pair such as d_1 and d_2 , we use LLM to compare their overall performance and finally choose the best-estimated directive point. In some cases, there may be an odd number of design points, we randomly select some design points and give these points a “bye” to a later round. This process continues until there are satisfied design points left. Note that the LLM used for initial sampling does not contain a fine-tuning process and even some closed LLMs such as ChatGPT can also be used in this stage.

3.3 Update LLM via Synthesis Feedback

To enhance the LLM's prediction and comparative ability, we fine-tune LLM based on the selected directive synthesis results in each sampling iteration. For each synthesis process, we generate related Verilog code and synthesis reports. We parse the report and extract associated PPA results and use these values as ground truth. Inspired by RLHF [69], we propose a novel fine-tuning strategy that learns from synthesis results. Figure 8 illustrates the fine-tuning framework for leveraging LLMs in HLS directive optimization. The process begins with constructing a comprehensive training dataset that correlates selected directives with their corresponding synthesis results. Each sample in the dataset comprises a raw code snippet, related directives, and the resulting PPA performance. These elements are carefully parsed and formatted to create directive prompts, which include contextual

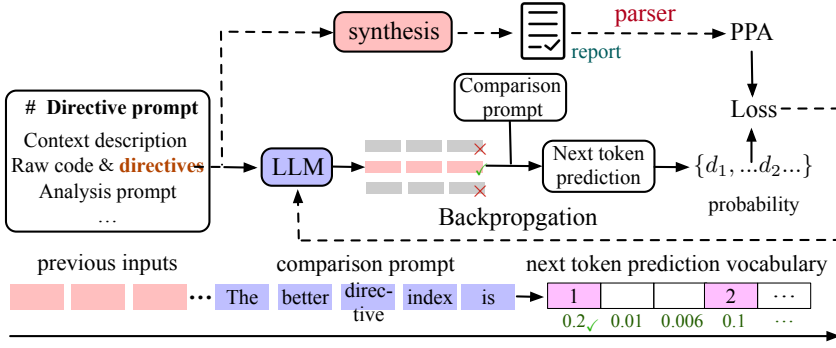


Fig. 8. Update LLM based on selected directives.

information, raw code, applied directives, and a tailored analysis prompt. These prompts are then used as inputs for the LLM. The LLM is then tasked with generating analysis thoughts, which are intermediate reasoning steps that provide insights into the relationship between directives and resulting PPA. Inspired by RLHF process, the framework generates multiple thoughts for each prompt. These thoughts are evaluated, and the most promising ones are selected to refine the LLM's generation policy. This selection process effectively reinforces the model's ability to reason accurately and generate high-quality predictions. To predict the optimal directive, the original input (including the raw code and directives), the selected analysis thoughts, and a comparison prompt are fed back into the LLM. The comparison prompt structures the task as a decision-making problem, prompting the LLM to predict which directive index yields better PPA improvements. The output is a probability distribution over possible directive indices, with higher probabilities assigned to directives likely to enhance PPA. The predicted directive is then evaluated through synthesis to update its PPA metrics. A parser extracts the updated PPA results, which is used to compute the objective function. By combining directive prompts, analysis thoughts, and feedback loops from synthesis results, this framework enables the LLM to learn to make data-driven decisions about which directives yield better results.

Thoughts Generation. By directive prompts and directives input, we can generate multiple thoughts. We use multinomial sampling to generate samples randomly. Define a sequence of input tokens $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$, the next token x_{n+1} is sampled from:

$$\begin{aligned} out &= \pi_{\theta}(\mathbf{x}), \\ pr &= \text{softmax}_T(out), \\ x_{n+1} &\sim \text{Categorical}(pr), \end{aligned} \quad (9)$$

where out is the next token logits output. softmax_T means the softmax function with temperature where the formula of logits out_i is $\frac{e^{out_i/T}}{\sum_j e^{out_j/T}}$. T is the temperature parameter where higher T makes the output distribution more uniform, thus introducing more randomness. *Categorical* means Categorical distribution. For example, assume the next token probability distribution is $\{dog : 0.4, cat : 0.5, bike : 0.1\}$, then the next token is selected according to their probabilities. We can generate K thoughts by this sampling strategy.

After thoughts generation, we combine original inputs, comparison prompts with multiple thoughts to LLM and obtain associated better directive prediction. Note that the end of the comparison prompt contains the question such as “the better directive index is”. Therefore, we can obtain certain outputs based on the next token prediction manner of LLM. The ground truth state is based

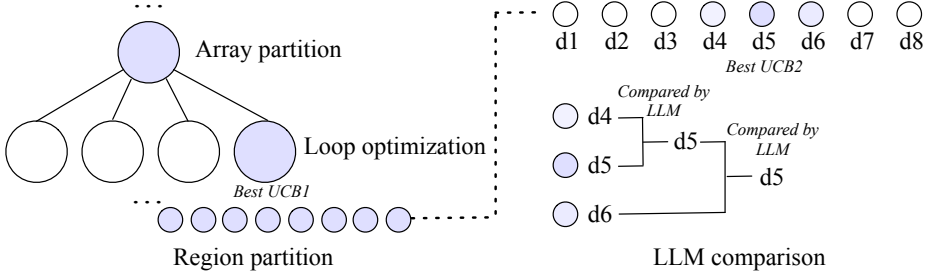


Fig. 9. Two-stage sampling in exploration flow.

on the synthesis result and we can obtain the probabilities of the directive index via the softmax function. We select the thought with the highest correct directive index probability and fine-tune LLM with two specialized loss functions. The training process can be expressed as follows:

$$\begin{aligned}
 th &= \{\pi_\theta(x, p_{t1}), \pi_\theta(x, p_{t1}, th_1), \dots, \pi_\theta(x, p_{t1}, th_1, \dots, th_{n-1})\}, \\
 out &= \pi_\theta(x, p_t, th), \\
 pr &= \text{softmax}(out), \\
 \mathcal{L} &= -\mathbb{E}_{x \sim D}(\log(\sigma(out_{d^*} - out_d)) + y_{d^*} \log(pr_{d^*})),
 \end{aligned} \tag{10}$$

where th is the “thoughts” outputs generated by LLM π_θ with input x and related prompt p_t autoregressively. out is the next token prediction given the comparison prompt. d^* is the ground truth directive index derived by synthesis results. d is the other directive index. The first loss function is a contrastive-like loss function where we aim at maximizing the logits output of d^* while minimizing the other index d . σ is the sigmoid function. The second loss is the conventional cross-entropy loss function. All x are sampled from selected directives D .

3.4 Two-Stage Sampling

Figure 9 shows the two-stage sampling process in exploration flow. In the first stage, we adopt a classical BO for sampling along with two acquisition functions across regions. The LLM is utilized for further comparison and selection in the second stage.

Sampling via Bayesian Optimization. In the first stage, the exploration flow is composed of a surrogate model (i.e., Gaussian process) and two different acquisition functions, alongside LLM for further selection. We tackle the search space with a similar hierarchical partition rule as previously mentioned. We first choose the best-estimated region via Equation (11). Then, we select directives via Equation (13) from this region. The proposed sampling strategy aims to select the best-estimated samples from important regions by a coarse-to-fine process. Specifically, we utilize LLM to extract informative features of selected directives given specified prompts. The extracted features then are used as input to Gaussian process regression to update the corresponding mean and variance. We take advantage of mean, variance function and UCB algorithms to formulate acquisition functions to sample directive design points from coarse to fine. Denote LLM as π_θ , raw inputs $\mathbf{x}$, prompts as $\mathbf{p}_t$. The directive features can be obtained by: $\mathbf{z}_x = \pi_\theta(\mathbf{x}, \mathbf{p}_t)$, where $\mathbf{z}_x$ is the hidden features of penultimate layer of LLM, which are then used to update GP with synthesized results. Based on GP, the first acquisition function is

$$UCB_1(\mathbf{x}) = \mu_1(\mathbf{x}) + \beta\sigma_1(\mathbf{x}), \tag{11}$$

ALGORITHM 1: Directives Design Optimization Flow

```

1: Input: LLM  $\pi_\theta$ , trials nums  $N$ , partitioned HLS space  $S = \{(\mathbf{x}_1^1, \mathbf{x}_2^1, \dots, \mathbf{x}_n^1), \dots, (\mathbf{x}_1^r, \mathbf{x}_2^r, \dots, \mathbf{x}_n^r)\}$ , pre-defined prompts  $\mathbf{p}_t$ , GP model as  $GP$ , acquisition functions  $UCB_1$  and  $UCB_2$ .
2: Output: Pareto set PS.
3: Initialize PS  $\leftarrow \emptyset$ ;
4: Initial set  $\{\mathbf{x}_*^1, \mathbf{x}_*^2, \dots, \mathbf{x}_*^r\} \leftarrow$  Initial sampling; ▷ Section 3.2
5: Performance  $\{\mathbf{y}_*^1, \mathbf{y}_*^2, \dots, \mathbf{y}_*^r\} \leftarrow$  Run FPGA tool;
6: for  $t \leftarrow 1$  to  $N$  do
7:   Updated LLM  $\pi_\theta \leftarrow$  Tuning on  $\{\mathbf{x}_*, \mathbf{y}_*\}$  pairs; ▷ Section 3.3
8:   Directive features extraction:  $\mathbf{z}_{\mathbf{x}_*} \leftarrow \pi_\theta(\mathbf{x}_*, \mathbf{p}_t)$ ;
9:    $\mu_1(\mathbf{x}), \sigma_1(\mathbf{x}) \leftarrow GP$  ▷ Predicted mean and var of each region
10:   $\mathbf{r}_* \leftarrow \operatorname{argmax} UCB_1(\mu_1(\mathbf{x}), \sigma_1(\mathbf{x}))$ ;
11:   $\mu_2(\mathbf{x}^{r*}), \sigma_2(\mathbf{x}^{r*}) \leftarrow GP$  ▷ Predicted mean and var within region  $\mathbf{r}_*$ 
12:   $\mathbf{x}'_* \leftarrow \operatorname{top-k}(UCB_2(\mu(\mathbf{x}^{r*}), \sigma(\mathbf{x}^{r*})))$ ;
13:   $\mathbf{x}'_* \leftarrow$  LLM compare and select from  $\mathbf{x}'_*$ ;
14:   $\mathbf{x}_* \leftarrow \mathbf{x}_* \cup \mathbf{x}'_*$  ▷ add to candidates
15: end for
16: Return Pareto set PS  $\leftarrow$  Optimal points from  $\mathbf{x}_*$ .

```

where $\mu_1(\mathbf{x})$ is the expected reward of each pre-defined region, which is obtained by taking the average of the expected values of points across the regions. β is the hyper-parameters to balance exploration and exploitation. $\sigma_1(\mathbf{x})$ is uncertainty function, which is defined as

$$\sigma_1(\mathbf{x}) = \sqrt{\frac{2 \log(T)}{n(\mathbf{x})}}, \quad (12)$$

where T is the trial number. $n(\mathbf{x})$ is the number of trials the region including $\mathbf{x}$. Equation (11) is a classical UCB algorithm that can be derived by Hoeffding's inequality [13]. As $n(\mathbf{x})$ increases, the uncertainty of related regions decreases. This acquisition function encourages exploration with less sampled regions with a logarithmic regret growth. After the selection of regions, we need to select the best-estimated directive design points from this region. We adopt a variant from the family of UCB algorithms proposed by Reference [70] as acquisition functions, which is defined as

$$UCB_2 \mathbf{x} = \mu_2(\mathbf{x}) + \lambda \sigma_2(\mathbf{x}), \quad (13)$$

where $\mu_2(\mathbf{x})$ represents the expected value of the function at any point $\mathbf{x}$, λ is a scalar to give a tradeoff between exploration and exploitation. $\sigma_2(\mathbf{x})$ is the standard deviation output by GP. Based on the Equation (13), we can select top k directive design points.

Sampling via LLM. Then, the selected points are evaluated via the tournament selection process as shown in Figure 9, where each pair is compared by LLM. Finally, we obtain a desired subset of design points.

Algorithm 1 demonstrates the whole design optimization flow. The algorithm commences by initializing necessary inputs which include the LLM model π_θ , a predetermined number of trials N , HLS design space S and pre-defined prompts $\mathbf{p}_t$, a Gaussian Process (GP) model equipped with UCB-based acquisition functions. The primary output of this algorithm is the Pareto set PS. We begin by setting the Pareto set PS to an empty state. Initial design points are strategically sampled using the LLM, which provides a data-driven foundation for subsequent optimization steps. This initial sampling is crucial as it populates the design space with viable starting points for exploration and is detailed further in Section 3.3. Next, performance metrics for these design points are assessed using FPGA tools. This evaluation is vital as it establishes the effectiveness of

each directive configuration under the current synthesis conditions. The core of the algorithm lies in its iterative loop, running for N trials, where the LLM is continuously updated and refined to enhance its predictive and comparative analysis capabilities. Each trial involves updating the LLM with new data pairs from design configurations and their corresponding performance outcomes. Directive features are extracted from the updated LLM for each design point. These features inform the GP model, which then predicts potential improvements in the design space. Utilizing UCB_1 , the algorithm selects promising regions within the HLS space. Within these regions, UCB_2 identifies the top-k design points based on the GP's predictions. The top-k points are further scrutinized by the LLM, ensuring that only the most promising configurations are considered for final selection.

4 Experimental Results

4.1 Experimental Settings

We conduct our experiments on seven benchmarks from open-source FPGA application benchmark polybench [71] including atax, bigc, gemm, gesummv, k3mm, syr2k, and syrk. These benchmarks contain several for-loops, matrix computations and memory communications. We generate around 2,000 points for each benchmark and parse the files to get related objectives. Numerous studies [72–77] have explored leveraging features from various stages of the synthesis process. Unlike these approaches, our work specifically concentrates on primitive-level transformations, which occur at the top level of design abstraction and do not involve intermediate representation (IR) level transformations. For instance, ScaleHLS [75] utilizes a DSE engine operating on multi-level IR to facilitate efficient DSE. In the broader context of related works, the work [72] adopts a transfer learning approach to import knowledge from a source domain to a target domain. Although this is not directly related to our work, it provides valuable insights and has inspired us to consider using target datasets as pretrained datasets for training a comprehensive HLS model. Additionally, the study [73] employs additional features extracted from dataflow graphs and incorporates a **Graph Neural Network (GNN)** to model these features. In this research, node attributes are derived from LLVM IR and Clang AST, showcasing a different method of feature utilization that enriches the understanding and capabilities of HLS tools. Since this work mainly considers directive design optimization, we do not take these works into consideration. We mainly compare our DSE results with some directive design optimization algorithms such as [12]. The target FPGA board is Xilinx xc7z030sbg485-1. We employ an open-source Llama-13B [23] as our LLM model. We leverage DeepSpeed and FlashAttention [78] for efficient LLM inference. The max tokens length is set to 2048. We use 4 NVIDIA GPU A100 80G for fine-tuning and inference. The synthesis process typically requires several minutes to generate a single-point synthesis report, which is the major bottleneck in DSE process. On the other hand, the runtime of LLM is considerably shorter than the synthesis time, it only takes several seconds to finish one directives comparison.

For objective selection, we consider the utilization of **look-up table (LUT)** as the primary area metric. LUTs are primarily used to implement control logic and arithmetic operations in FPGA designs. For tasks that demand high parallelism or complex computational logic, LUT utilization frequently becomes a critical resource indicator, reflecting the complexity and efficiency of the resulting design. We acknowledge that other FPGA resources, such as BRAMs, DSPs, and FFs, also significantly contribute to resource usage. Some previous works (e.g., [73, 74]) similarly omit BRAM usage due to their specific HLS DSE settings, where the primary focus lies in leveraging GNNs for HLS DSE task. We will include these metrics into future work. Power as a metric is directly extracted from the synthesis report. We choose the product of latency and clock period as the delay metric. Consequently, our overall performance metric, denoted as PPA, is defined as: $PPA = \alpha_1 \cdot Power + \alpha_2 \cdot LUT + Delay$. Incorporating additional resources explicitly into the evaluation framework represents a valuable extension for future work. As with DSE tasks, we

Table 1. Normalized ADRS Results

Model	Normalized ADRS					
	FPL18 [80]	ANN	BT	DAC19 [79]	TODAES22 [12]	Ours
atax	0.63	1.00	0.78	1.12	0.43	0.24
bicg	0.89	1.00	0.94	1.58	0.82	0.49
gemm	0.91	1.00	1.06	0.98	0.82	0.53
gesummv	0.94	1.00	1.12	1.08	0.78	0.54
k3mm	1.08	1.00	1.16	1.26	0.84	0.73
syr2k	0.81	1.00	0.92	1.21	0.76	0.68
syrk	0.78	1.00	0.88	1.02	0.66	0.39
Average	0.86	1.0	0.98	1.18	0.73	0.51

use around 5% of online data samples. The number of initialized sampling points is based on the partition regions of each benchmark. For the partition strategy, we mainly consider partitioning the design space based on loop optimization. Utilizing some clustering algorithms or hierarchical multi-stage region partitioning may bring extra improvement, and we leave it in future works. We utilize the **average distance to reference set (ADRS)** as our evaluation metric. The ADRS metric quantifies the average distance between solutions in the learned Pareto optimal set and the corresponding points in the true (or “real”) Pareto optimal set [79]. Formally, ADRS is defined as follows:

$$ADRS(\Gamma, \Omega) = \frac{1}{|\Gamma|} \sum_{\gamma \in \Gamma} \min_{\omega \in \Omega} f(\gamma, \omega), \quad (14)$$

where Ω represents the learned Pareto set obtained by the algorithm, Γ denotes the true Pareto optimal set serving as a reference, and $f(\gamma, \omega)$ measures the distance between two points, with $\gamma \in \Gamma$ and $\omega \in \Omega$. The term $|\Gamma|$ indicates the number of points in the true Pareto set. A lower ADRS value indicates that the learned Pareto set is closer to the ideal (true) Pareto set, thus reflecting better approximation and higher solution quality. In this context, the observed 15% improvement in the normalized ADRS metric is significant. Even modest reductions in ADRS can correspond to substantial practical improvements in solution quality, as they imply solutions that are closer in performance to the theoretically optimal frontier.

4.2 Results and Analysis

Table 1 demonstrates the statistical results on normalized ADRS, while we expressed them as ratios to the results of ANN. ANN stands for artificial neural network, as detailed in Reference [10]. We compare our method with several classical algorithms. Among them, TODAES22 [12] uses the Bayesian method and Gaussian process, and build a non-linear multi-fidelity model to capture the non-linear correlation between different fidelity. FPL18 [80] is also based on BO and Gaussian process, however, it builds linear multi-fidelity and independent multi-objective models. Besides, the methods based on ANN and Boosting tree (short for BT) are also introduced in References [10, 11]. In addition, Reference [79], abbreviated as DAC19, defines several regression models that guide HLS designs with existing designs. Our method demonstrates the lowest ADRS values across all benchmarks, with an average normalized ADRS of 0.51, significantly outperforming competing approaches such as FPL18 (0.86), ANN (1.00), BT (0.98), DAC19 (1.18), and TODAES22 (0.73). This improvement translates to a 15% reduction in ADRS compared with the second-best method, TODAES22 [12]. Table 2 presents the normalized standard deviations of the ADRS across various models for different benchmarks. It distinctly shows that our method consistently achieves the

Table 2. Normalized Std of ADRS Results

Model	Normalized Standard Deviation of ADRS					
	FPL18 [80]	ANN	BT	DAC19 [79]	TODAES22 [12]	Ours
atax	0.34	1.00	0.24	0.92	0.13	0.05
bicg	0.46	1.00	0.37	1.06	0.31	0.10
gemm	0.26	1.00	0.02	0.26	0.09	0.02
gesummv	0.66	1.00	0.45	1.26	0.25	0.04
k3mm	0.34	1.00	0.16	0.88	0.27	0.02
syr2k	0.45	1.00	0.04	1.14	0.27	0.10
syrk	0.32	1.00	0.28	0.88	0.12	0.02
Average	0.41	1.00	0.22	0.91	0.20	0.05

lowest standard deviations, indicating superior stability in performance when compared with other approaches such as FPL18, ANN, BT, DAC19, and TODAES22. Specifically, our approach yields the most stable results across all benchmarks, with notable reductions in variability. For instance, in the ‘gesummv’ benchmark, our method reduces the standard deviation to 0.04 compared with 1.00 for the ANN model, demonstrating a significant improvement in reliability and predictability of performance outcomes. The reduced standard deviations across all cases underscore the robustness of our method, suggesting that it not only enhances performance but also ensures consistent results across different runs and scenarios, making it a highly reliable solution for HLS DSE.

Figure 10 illustrates the learned Pareto points for all benchmarks, highlighting the tradeoffs between power and delay objectives. The scatter plots clearly show that the Pareto points generated by our proposed methodology align more closely with the real Pareto front (red crosses) compared with traditional approaches such as TODAES22, DAC19, and FPL18. This visual proximity to the ideal Pareto front demonstrates the capability of our method to effectively optimize power and delay simultaneously. For the Gesummv benchmark (Figure 10 (d)), our learned Pareto points cluster significantly closer to the real Pareto front, indicating a superior balance between power reduction and delay improvement. Specifically, our approach achieves a power reduction of up to 12% and a delay reduction of approximately 8% compared with competing methods. This efficiency highlights the ability of our methodology to explore the design space in a way that prioritizes energy savings without compromising performance. In the Syr2k benchmark (Figure 10 (f)), our Pareto points also exhibit a strong alignment with the real Pareto front. The results indicate a 10% reduction in power and a 6% decrease in delay, suggesting that our method is capable of optimizing both objectives concurrently. The clustering of our Pareto points closer to the ideal front compared with TODAES22 and DAC19 reinforces the robustness of our approach in handling multi-objective optimization. For other benchmarks such as Atax (Figure 10 (a)) and Gemm (Figure 10 (c)), the graphical representation further illustrates the effectiveness of our methodology. In these cases, our Pareto points consistently dominate those of competing methods, achieving a better tradeoff between power and delay. For instance, in the Gemm benchmark, our Pareto points exhibit a clear reduction in both objectives compared with TODAES22 and FPL18, showcasing how our approach efficiently explores the directive space. In the K3mm (Figure 10 (e)) and Syrk (Figure 10 (g)) benchmarks, our method maintains its effectiveness. Although the improvements in these cases are more modest, our Pareto points are still closer to the real Pareto front than those produced by traditional methods. This consistency across benchmarks highlights the adaptability of our approach to varying design challenges in HLS.

Figure 11 compares the directive comparison accuracy across different language models, including Llama-13B, ChatGPT, and our fine-tuned LLM enhanced by the Learning-from-Synthesis approach.

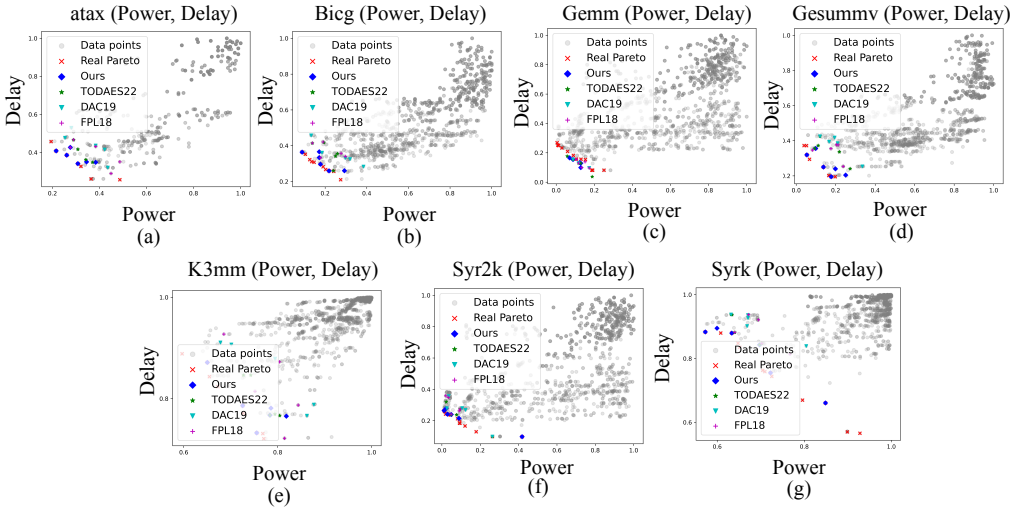


Fig. 10. Learned DSE results: (a) atax; (b) Bicg; (c) Gemm; (d) Gesummv; (e) K3mm; (f) Syr2k; (g) Syrk;

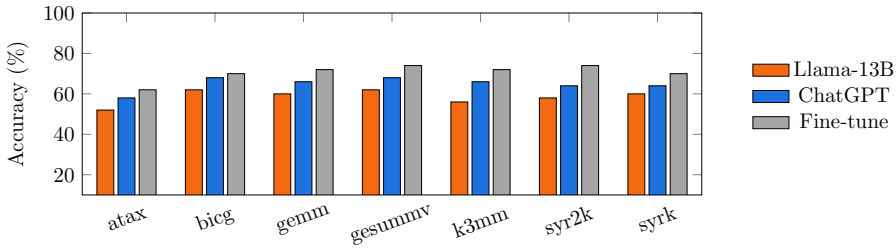


Fig. 11. Directive comparison accuracy between different language models.

The comparison highlights the varying capabilities of these models in accurately predicting better directives for DSE. Llama-13B and ChatGPT represent models that have not undergone fine-tuning using synthesis results. Among these, ChatGPT demonstrates a notable improvement over the open-source model Llama-13B, achieving higher accuracy across all benchmarks. This suggests that ChatGPT's inherent pre-training and prompt-based reasoning capabilities offer a stronger baseline for directive decision-making. However, the results also reveal the limitations of relying solely on pre-trained models and prompt engineering for DSE tasks. Our fine-tuned LLM, which incorporates the Learning-from-Synthesis approach, achieves a significant accuracy improvement over both Llama-13B and ChatGPT across all benchmarks. The fine-tuned model consistently outperforms ChatGPT by margins ranging from 4% to 10%, depending on the benchmark. These results emphasize the strength of our Learning-from-Synthesis approach, where the fine-tuning process leverages synthesis results to guide the model toward better directive selection. This additional training step enables the model to account for nuanced correlations between directives and their impact on PPA. Figure 12 illustrates the clustering visualization results between deep kernel features [12] and LLM-extracted features. We can observe that the LLM-extracted features are more scattered, which demonstrates that the LLM-extracted features are more informative and distinguishable.

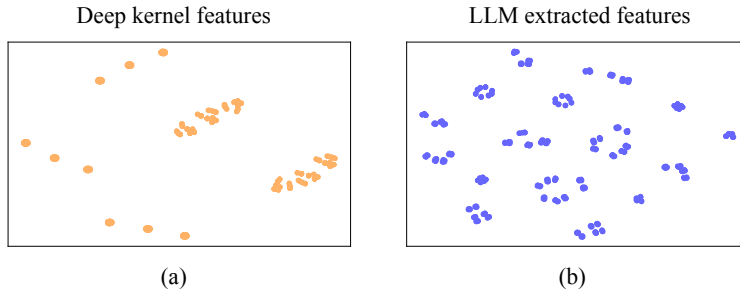


Fig. 12. t-SNE visualization results: (a) Deep kernel features result; (b) LLM-extracted features result.

4.3 Discussions

Discussion on Prompt Engineering. Prompt engineering such as chain-of-thoughts [68] plays an important role in LLM. This article considers different types of prompts including directive analysis, directive comparison, PPA analysis, and so on. To maintain simplicity, we adopt a simple strategy where only one thought response is combined into the subsequent inference pipeline. While more advanced prompt engineering techniques like tree-of-thoughts or algorithm-of-thoughts may offer improved performance, they often require additional computational resources.

Discussion on Complicated HLS designs. Our framework is currently not specifically designed to handle extremely complex HLS designs, but we are actively exploring solutions in two strategic areas to address this limitation. Firstly, the ability of our system to process complex designs is highly dependent on the input length of the HLS code. Modern LLMs are equipped with sophisticated techniques to manage long sequence inferences-capable of processing up to 128k tokens or more. This capability allows them to effectively manage the intricacies of complex HLS designs. Moreover, these designs often comprise several smaller, modular components that can be addressed separately. This modularity enables LLMs to break down complex tasks into more manageable parts, significantly enhancing the practicality of using LLMs for intricate HLS tasks. Secondly, the continuous advancements in LLMs, exemplified by the upcoming introduction of the llama3-400B, are set to greatly enhance their ability to handle more complex and varied scenarios. This evolution is crucial as traditional learning-based methods frequently encounter difficulties with complex design spaces, primarily due to their limited ability to adapt and scale. In contrast, LLM-based approaches are not only equipped with advanced comprehension capabilities but also demonstrate robust predictive abilities. These features allow them to excel where traditional methods falter, offering a promising avenue for navigating and optimizing complex HLS designs. Additionally, the integration of techniques such as transfer learning and model fine-tuning could further amplify the effectiveness of LLMs in this domain. By leveraging pre-trained models that are subsequently adapted to specific HLS tasks, our framework can better understand and predict the nuances of different design paradigms. This approach not only saves computational resources but also shortens the development cycle, making the application of LLMs to complex HLS designs not just viable but highly efficient. With these strategies, our framework is poised to transform the landscape of HLS design optimization, making it more accessible and effective for handling the increasing complexity of modern electronic systems.

In our pipeline, LLM primarily served as an efficient sampler, providing informed candidate solutions within the BO-based DSE. Notably, the computational complexity of LLM-based sampling remains largely independent of the HLS design complexity (e.g., more loops or modules). This is because candidate generation via LLM inference is essentially a constant-time operation per

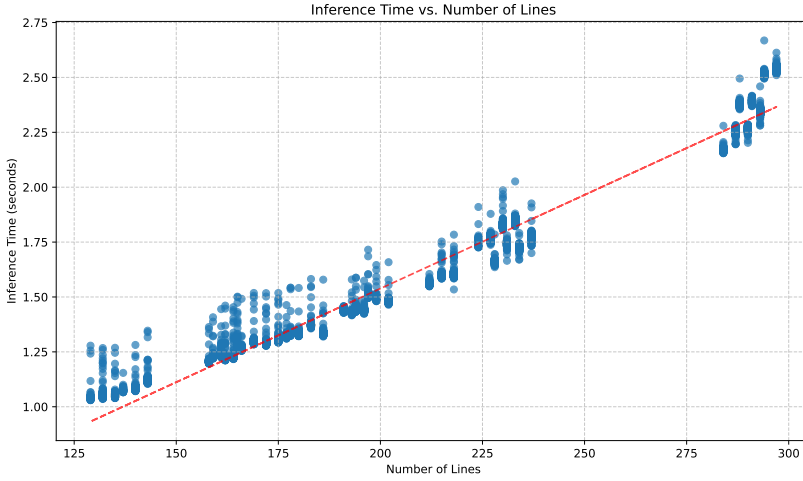


Fig. 13. Inference time vs. number of lines.

candidate, unaffected by the dimensionality or intricacy of the optimization problem. Additionally, BO is well-established in literature to scale effectively with increasing complexity. Specifically, BO typically exhibits linear or even sublinear complexity growth compared with exhaustive or naive methods, making it suitable for larger and more complex design spaces. Recent studies, such as [81] and [12], have empirically validated the efficiency and scalability of BO in handling complex optimization scenarios and larger hardware DSE benchmarks. We also evaluate the scalability of the proposed method by analyzing the computational cost across designs of varying complexity. As shown in Figure 13, our method achieves linear computational cost relative to design complexity.

5 Conclusion

This article presents a novel large language model (LLM)-based framework for optimizing high-level synthesis (HLS) directive design. Our approach leverages LLM's sophisticated directive comprehension capabilities to enhance DSE performance across multiple dimensions. To address the cold-start problem, we introduce an LLM-based initial sampling strategy. The framework implements a two-stage sampling pipeline that enables coarse-to-fine sampling, facilitating the identification of promising search regions. Additionally, we incorporate synthesis results to refine the LLM and employ the updated model for comparative analysis of directive efficacy. Comprehensive experimental results validate the effectiveness of our LLM-based methodology. Furthermore, this research offers fresh perspectives on utilizing LLMs in exploration tasks and incorporating synthesis feedback for model refinement. These contributions aim at catalyzing innovative research directions in the field of HLS.

References

- [1] Philippe Coussy, Daniel D Gajski, Michael Meredith, and Andres Takach. 2009. An introduction to high-level synthesis. *IEEE Design & Test* 26, 4 (2009), 8–17.
- [2] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. 2011. High-level synthesis for FPGAs: From prototyping to deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 30, 4 (2011), 473–491.
- [3] Daniel D Gajski, Nikil D Dutt, Allen CH Wu, and Steve YL Lin. 2012. *High-Level Synthesis: Introduction to Chip and System Design*. Springer Science & Business Media.

- [4] Jieru Zhao, Liang Feng, Sharad Sinha, Wei Zhang, Yun Liang, and Bingsheng He. 2019. Performance modeling and directives optimization for high-level synthesis on FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 39, 7 (2019), 1428–1441.
- [5] Wei Zuo, Yun Liang, Peng Li, Kyle Rupnow, Deming Chen, and Jason Cong. 2013. Improving high level synthesis optimization opportunity through polyhedral transformations. In *ACM International Symposium on Field-Programmable Gate Arrays (FPGA)*. 9–18.
- [6] Peng Li, Peng Zhang, Louis-Noel Pouchet, and Jason Cong. 2015. Resource-aware throughput optimization for high-level synthesis. In *ACM International Symposium on Field-Programmable Gate Arrays (FPGA)*. 200–209.
- [7] Jason Cong, Peng Zhang, and Yi Zou. 2012. Optimizing memory hierarchy allocation with loop transformations for high-level synthesis. In *The 49th Annual Design Automation Conference 2012, DAC'12, San Francisco, CA, USA, June 3-7, 2012*, Patrick Groeneveld, Donatella Sciuto, and Soha Hassoun (Eds.). ACM, 1233–238.
- [8] Yuxin Wang, Peng Li, Peng Zhang, Chen Zhang, and Jason Cong. 2013. Memory partitioning for multidimensional arrays in high-level synthesis. In *ACM/IEEE Design Automation Conference (DAC)*. 1–8.
- [9] Nan Wu, Hang Yang, Yuan Xie, Pan Li, and Cong Hao. 2022. High-level synthesis performance prediction using GNNs: Benchmarking, modeling, and advancing. In *ACM/IEEE Design Automation Conference (DAC)*. 49–54.
- [10] Steve Dai, Yuan Zhou, Hang Zhang, Ecenur Ustun, Evangeline FY Young, and Zhiru Zhang. 2018. Fast and accurate estimation of quality of results in high-level synthesis with machine learning. In *IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 129–132.
- [11] Ecenur Ustun, Shaojie Xiang, Jinny Gui, Cunxi Yu, and Zhiru Zhang. 2019. LAMDA: Learning-assisted multi-stage autotuning for FPGA design closure. In *IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 74–77.
- [12] Qi Sun, Tinghuan Chen, Siting Liu, Jianli Chen, Hao Yu, and Bei Yu. 2022. Correlated multi-objective multi-fidelity optimization for HLS directives design. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 27, 4 (2022), 1–27.
- [13] Peter Auer. 2002. Using confidence bounds for exploitation-exploration trade-offs. *Journal of Machine Learning Research (JMLR)* 3, Nov (2002), 397–422.
- [14] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günemann, Eyke Hüllermeier, et al. 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences* 103 (2023), 102274.
- [15] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *Comput. Surveys* 56, 2 (2023), 1–40.
- [16] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2023. A survey on large language model based autonomous agents. *arXiv preprint* (2023).
- [17] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *CoRR abs/2303.18223* (2023).
- [18] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Annual Conference on Neural Information Processing Systems (NeurIPS)* 32 (2019).
- [19] Noam Shazeer. 2020. Glue variants improve transformer. *arXiv preprint* (2020).
- [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Annual Conference on Neural Information Processing Systems (NeurIPS)* 30 (2017).
- [21] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint* (2019).
- [22] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Empirical Methods in Natural Language Processing (EMNLP)*.
- [23] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint* (2023).
- [24] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lelio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothee Lacroix, and William El Sayed. 2023. Mistral 7B. *CoRR abs/2310.06825* (2023).
- [25] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Ross Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, Bonita Bhaskaran, Bryan Catanzaro, Arjun Chaudhuri, Sharon Clay, Bill Dally, Laura Dang, Parikshit Deshpande, Siddhanth Dhodhi, Sameer Halepete, Eric Hill, Jiashang Hu, Sumit Jain, Bruce Khailany, Kishor Kunal, Xiaowei Li, Hao Liu, Stuart F. Oberman, Sujeet Omar, Sreedhar Pratty,

- Jonathan Raiman, Ambar Sarkar, Zhengjiang Shao, Hanfei Sun, Pratik P. Suthar, Varun Tej, Kaizhe Xu, and Haoxing Ren. 2023. ChipNeMo: Domain-adapted LLMs for chip design. *CoRR* abs/2311.00176 (2023).
- [26] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2023. Chip-Chat: Challenges and opportunities in conversational hardware design. In *ACM/IEEE Workshop on Machine Learning CAD (MLCAD)*. 1–6.
- [27] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. 2023. GPT4AIGChip: Towards next-generation AI accelerator design automation via large language models. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–9.
- [28] Zhuolun He, Haoyuan Wu, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2023. ChatEDA: A large language model powered autonomous agent for EDA. In *ACM/IEEE Workshop on Machine Learning CAD (MLCAD)*. IEEE, 1–6.
- [29] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2023. Benchmarking large language models for automated verilog RTL code generation. In *IEEE/ACM Proceedings Design, Automation and Test in Europe (DATE)*. IEEE, 1–6.
- [30] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2023. RTLCoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution. *arXiv preprint* (2023).
- [31] YunDa Tsai, Mingjie Liu, and Haoxing Ren. 2024. RTLfixer: Automatically fixing RTL syntax errors with large language model. In *ACM/IEEE Design Automation Conference (DAC)*.
- [32] Marcelo Orenes-Vera, Margaret Martonosi, and David Wentzlaff. 2023. Using LLMs to facilitate formal verification of RTL. *arXiv preprint* (2023), arXiv–2309.
- [33] Wenji Fang, Mengming Li, Min Li, Zhiyuan Yan, Shang Liu, Hongce Zhang, and Zhiyao Xie. 2024. AssertLLM: Generating and evaluating hardware verification assertions from design specifications via multi-LLMs. *CoRR* abs/2402.00386 (2024).
- [34] Rahul Kande, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Shailja Thakur, Ramesh Karri, and Jeyavijayan Rajendran. 2023. LLM-assisted generation of hardware assertions. *CoRR* (2023).
- [35] OpenAI. 2023. GPT-4 Technical Report. (2023). [arXiv:cs.CL/2303.08774](https://arxiv.org/abs/2303.08774)
- [36] Benjamin Carrion Schafer and Kazutoshi Wakabayashi. 2012. Divide and conquer high-level synthesis design space exploration. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 17, 3 (2012), 1–19.
- [37] B Carrion Schafer and Kazutoshi Wakabayashi. 2012. Machine learning predictive modelling high-level synthesis design space exploration. *IET Computers & Digital Techniques* 6, 3 (2012), 153–159.
- [38] Anushree Mahapatra and Benjamin Carrion Schafer. 2014. Machine-learning based simulated annealer method for high level synthesis design space exploration. In *Proceedings of the 2014 Electronic System Level Synthesis Conference (ESLsyn)*. IEEE, 1–6.
- [39] Lorenzo Ferretti, Giovanni Ansaloni, and Laura Pozzi. 2018. Cluster-based heuristic for high level synthesis design space exploration. *IEEE Transactions on Emerging Topics in Computing* 9, 1 (2018), 35–43.
- [40] Benjamin Carrion Schafer. 2015. Probabilistic multiknob high-level synthesis design space exploration acceleration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 35, 3 (2015), 394–406.
- [41] Charles Lo and Paul Chow. 2016. Model-based optimization of high level synthesis directives. In *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 1–10.
- [42] Stéphane Pouget, Louis-Noël Pouchet, and Jason Cong. 2025. Automatic hardware pragma insertion in high-level synthesis: A non-linear programming approach. *ACM Transactions on Design Automation of Electronic Systems* 30, 2 (2025), 1–44.
- [43] Florian Faissolle, George A Constantinides, and David Thomas. 2019. Formalizing loop-carried dependencies in coq for high-level synthesis. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 315–315.
- [44] Sakari Lahti, Panu Sjövall, Jarno Vanne, and Timo D Hämäläinen. 2018. Are we there yet? A study on the state of high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 5 (2018), 898–911.
- [45] M Akif Özkan, Arsène Pérard-Gayot, Richard Membarth, Philipp Slusallek, Roland Leißa, Sebastian Hack, Jürgen Teich, and Frank Hannig. 2020. AnyHLS: High-level synthesis with partial evaluation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 3202–3214.
- [46] Georgios Zacharopoulos, Andrea Barbon, Giovanni Ansaloni, and Laura Pozzi. 2018. Machine learning approach for loop unrolling factor prediction in high level synthesis. In *2018 International Conference on High Performance Computing & Simulation (HPCS)*. IEEE, 91–97.
- [47] Mohammad Riazati, Masoud Daneshdatab, Mikael Sjödin, and Björn Lisper. 2022. DeepFlexiHLS: Deep neural network flexible high-level synthesis directive generator. In *2022 IEEE Nordic Circuits and Systems Conference (NorCAS)*. IEEE, 1–6.
- [48] Xufeng Yao, Jiaxi Jiang, Yuxuan Zhao, Peiyu Liao, Yibo Lin, and Bei Yu. 2025. Evolution of optimization algorithms for global placement via large language models. *CoRR* abs/2504.17801 (2025).

- [49] Lin Chen, Ran Chen, Shoubo Hu, Xufeng Yao, Zhentao Tang, Shixiong Kai, Siyuan Xu, Mingxuan Yuan, Jianye Hao, Bei Yu, et al. 2025. PCBAgent: An agent-based framework for high-density printed circuit board placement. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 781–787.
- [50] Zhuolun He, Yuan Pu, Haoyuan Wu, Yuhang Qiu, Tairu Qiu, Bei Yu, et al. 2025. Large language models for EDA: From assistants to agents. *Foundations and Trends® in Electronic Design Automation* 14, 4 (2025), 295–314.
- [51] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. 2025. Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 300–307.
- [52] Chia-Tung Ho, Jing Gong, Yunsheng Bai, Chenhui Deng, Haoxing Ren, and Brucek Khailany. 2025. Marco: Configurable graph-based task solving and multi-AI agents framework for hardware design. *CoRR abs/2504.01962* (2025).
- [53] Chia-Tung Ho and Haoxing Ren. 2024. Large language model (LLM) for standard cell layout design optimization. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–6.
- [54] Zhe Xiao, Xu He, Haoyang Wu, Bei Yu, and Yang Guo. 2025. EDA-Copilot: A RAG-powered intelligent assistant for EDA tools. *ACM Transactions on Design Automation of Electronic Systems* (2025).
- [55] Bei Yu. 2023. Machine learning in EDA: When and how. In *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 1–6.
- [56] Yuan Pu, Zhuolun He, Tairu Qiu, Haoyuan Wu, and Bei Yu. 2024. Customized retrieval augmented generation and benchmarking for EDA tool documentation QA. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [57] Xufeng Yao, Haoyang Li, Tsz Ho Chan, Wenyi Xiao, Mingxuan Yuan, Yu Huang, Lei Chen, and Bei Yu. 2024. HDLdebugger: Streamlining HDL debugging with large language models. *arXiv preprint arXiv:2403.11671* (2024).
- [58] Xufeng Yao, Yiwen Wang, Xing Li, Yingzhao Lian, Ran Chen, Lei Chen, Mingxuan Yuan, Hong Xu, and Bei Yu. 2024. RTLRewriter: Methodologies for large models aided RTL code optimization. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–7.
- [59] Yingbing Huang, Lily Jiaxin Wan, Hanchen Ye, Manvi Jha, Jinghua Wang, Yuhong Li, Xiaofan Zhang, and Deming Chen. 2024. New solutions on LLM acceleration, optimization, and application. In *ACM/IEEE Design Automation Conference (DAC)*. 1–4.
- [60] Lily Jiaxin Wan, Hanchen Ye, Jinghua Wang, Manvi Jha, and Deming Chen. 2024. An iteratively-refined dataset for high-level synthesis functional verification through LLM-aided bug injection. In *IEEE International Workshop on LLM-Aided Design (LAD)*. IEEE, 1–6.
- [61] Kangwei Xu, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, Ulf Schlichtmann, and Bing Li. 2024. Automated C/C++ program repair for high-level synthesis via large language models. In *ACM/IEEE Workshop on Machine Learning CAD (MLCAD)*. 1–9.
- [62] Chenwei Xiong, Cheng Liu, Huawei Li, and Xiaowei Li. 2024. HLLSPilot: LLM-based high-level synthesis. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [63] Haocheng Xu, Haotian Hu, and Sitao Huang. 2024. Optimizing high-level synthesis designs with retrieval-augmented large language models. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–5.
- [64] Sneha Swaroopa, Rijoy Mukherjee, Anushka Debnath, and Rajat Subhra Chakraborty. 2024. Evaluating large language models for automatic register transfer logic generation via high-level synthesis. *arXiv:2408.02793*. Retrieved from <https://arxiv.org/abs/2408.02793>
- [65] Yuchao Liao, Tosiron Adegbiya, and Roman Lysecky. 2024. Are LLMs any good for high-level synthesis? *arXiv preprint arXiv:2408.10428* (2024).
- [66] James T Meech. 2023. Leveraging high-level synthesis and large language models to generate, simulate, and deploy a uniform random number generator hardware design. *arXiv preprint arXiv:2311.03489* (2023).
- [67] Ali Emre Oztas and Mahdi Jelodari. 2024. Agentic-HLS: An agentic reasoning based high-level synthesis system using large language models. *arXiv preprint arXiv:2412.01604* (2024).
- [68] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.).
- [69] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Annual Conference on Neural Information Processing Systems (NeurIPS)* 35 (2022), 27730–27744.
- [70] Niranjan Srinivas, Andreas Krause, Sham M. Kakade, and Matthias W. Seeger. 2010. Gaussian process optimization in the bandit setting: No regret and experimental design. In *International Conference on Machine Learning (ICML)*. 1015–1022.

- [71] Louis-Noël Pouchet and Scott Grauer-Gray. 2012. Polybench: The polyhedral benchmark suite. <http://www-roc.inria.fr/~pouchet/software/polybench>. (2012).
- [72] Lorenzo Ferretti, Jihye Kwon, Giovanni Ansaloni, Giuseppe Di Guglielmo, Luca P Carloni, and Laura Pozzi. 2020. Leveraging prior knowledge for effective design-space exploration in high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 39, 11 (2020), 3736–3747.
- [73] Lorenzo Ferretti, Andrea Cini, Georgios Zacharopoulos, Cesare Alippi, and Laura Pozzi. 2022. Graph neural networks for high-level synthesis design space exploration. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 28, 2 (2022), 1–20.
- [74] Nan Wu, Yuan Xie, and Cong Hao. 2022. IronMan-Pro: Multiobjective design space exploration in HLS via reinforcement learning and graph neural network-based modeling. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 42, 3 (2022), 900–913.
- [75] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. 2022. ScaleHLS: A new scalable high-level synthesis framework on multi-level intermediate representation. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 741–755.
- [76] Hanchen Ye, David Z Pan, Chris Leary, Deming Chen, and Xiaoqing Xu. 2024. Subgraph extraction-based feedback-guided iterative scheduling for HLS. In *IEEE/ACM Proceedings Design, Automation and Test in Europe (DATE)*. IEEE, 1–6.
- [77] Weichuang Zhang, Jieru Zhao, Guan Shen, Quan Chen, Chen Chen, and Minyi Guo. 2024. An optimizing framework on MLIR for efficient FPGA-based accelerator generation. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 75–90.
- [78] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Annual Conference on Neural Information Processing Systems (NeurIPS)* 35 (2022), 16344–16359.
- [79] Shuangnan Liu, Francis CM Lau, and Benjamin Carrion Schafer. 2019. Accelerating FPGA prototyping through predictive model-based HLS design space exploration. In *ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [80] Charles Lo and Paul Chow. 2018. Multi-fidelity optimization for high-level synthesis directives. In *IEEE International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 272–277.
- [81] Jasper Snoek, Oren Rippel, Kevin Swersky, Ryan Kiros, Nadathur Satish, Narayanan Sundaram, Mostofa Patwary, Mr Prabhat, and Ryan Adams. 2015. Scalable bayesian optimization using deep neural networks. In *International Conference on Machine Learning*. PMLR, 2171–2180.

Received 18 March 2024; revised 16 March 2025; accepted 21 April 2025